

## Task 0

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
DH Parameters: (3072 bit)
GROUP: ffdhe3072

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3

(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

## Task 1

```
(.venv) ubuntu@ubuntu2204:~/csce465-agentsec/hw2$ python baseline_ctr.py
Original Plaintext: {"action":"READ","path":"notes.txt"}
Original Ciphertext: 6c88d223af56549b4db3ca54cf22679a15aa00327a30e9af96afca45b3582c34fb467f7e

--- XOR Bit-Flipping Analysis ---
Original Text: READ -> Bytes: [82, 69, 65, 68]
Target Text:   EXEC -> Bytes: [69, 88, 69, 67]
XOR Mask:      ['0x17', '0x1d', '0x04', '0x07']
-----
Index 11 | Original CT Byte: 0x54 ^ Mask: 0x17 => Modified CT Byte: 0x43
Index 12 | Original CT Byte: 0xcf ^ Mask: 0x1d => Modified CT Byte: 0xd2
Index 13 | Original CT Byte: 0x22 ^ Mask: 0x04 => Modified CT Byte: 0x26
Index 14 | Original CT Byte: 0x67 ^ Mask: 0x07 => Modified CT Byte: 0x60

--- [Transmission 1] Receiver Processing Tampered Message ---
[Receiver] Successfully processed command #1: {'action': 'EXEC', 'path': 'notes.txt'}

--- [Transmission 2] Replay Attack Executed ---
[Receiver] Successfully processed command #2: {'action': 'EXEC', 'path': 'notes.txt'}
```

The code simulates an unauthenticated client server system that is encrypted using AES-CTR without message authentication. The sender encrypts a JSON payload using AES-256-CTR with a 128-bit random nonce and a secret key, and the relay simulating an attacker intercepts the ciphertext mid transfer. The simulated attacker does not have the encryption key, but it knows the fixed format and position of the action field which in this case is *READ* starting at offset 11. There is also a receiver that decrypts the incoming ciphertext using the preshared AES key, parses the JSON text, and executes the command. In my experiment, the relay successfully executed the bit-flipping integrity attack by calculating the XOR mask between *READ* and *EXEC*, which resulted in the mask. XORing the mask directly into ciphertext positions 11-14 caused the receiver to decrypt the tampered payload without triggering any decryption errors. The relay executed a replay attack by capturing and retransmitting the modified ciphertext a second time using the same nonce, leading to the receiver decrypting and processing command #2 identically to command #1, failing to detect that the message was a duplicated transmission. The root cause of the vulnerabilities is a lack of ciphertext malleability protection due to AES-CTR operating as a stream cipher where the ciphertext byte is generated by XORing the plaintext with the keystream. This allows the ciphertext byte to be modified with an XOR delta that yields  $(C_i \oplus \Delta) \oplus K_i = P_i \oplus \Delta$  when decrypted. As a result, unauthenticated stream ciphers are shown to be fully malleable and highlight that encryption alone guarantees confidentiality without providing any integrity verification.

## Task 2

The design in *handshake.py* pulls from concepts in cryptography from the textbook by combining Diffie-Hellman key establishment with long-term RSA-PSS digital signatures to achieve authentication, replay resistance, forward secrecy, and key separation as discussed in lecture. Requiring each party to sign a SHA-256 hash of a length-prefixed canonical transcript allows the system to prevent active Man-in-the-Middle and reflection attacks. This works by making sure any payload modification in transit invalidates the signature verification, while role-prefixed signing and identity positioning stop an attacker from echoing a party's own messages back to themselves. The replay resistance is guaranteed because fresh nonces ensure a unique transcript hash for every session, while forward secrecy is achieved by discarding ephemeral DH private keys immediately after deriving the shared secret. Domain-separated HMAC-SHA-256 derivations also prevent key re-use vulnerabilities across communication directions. If the digital signatures were removed, the protocol would be downgraded into anonymous Diffie-Hellman, completely removing entity authentication and allowing an active adversary to intercept public values, establish separate shared keys with both endpoints, and transparently read or manipulate all session traffic. This is why signatures play a key role in the design, allowing parties to be identified properly, maintaining integrity and ensuring communication occurs with the intended party.

## Task 4

### Security note

Our system works under the Delov Yao threat model by assuming an active network adversary who has full control over the transit channel. The attacker is able to intercept, modify, reorder, replay, or inject network frames. They can also attempt cross channel reflection attacks. Non goals of the protocol include defending against physical hardware side channel attacks, mitigating network level denial of service flood, or preserving secrecy if one of the endpoint's long term RSA private key is exfiltrated.

AES-CTR operates by transforming a block cipher into a stream cipher and encrypting sequential counter values concatenated with an initialization vector to create a pseudorandom keystream. The ciphertext is generated by doing an XOR operation with the plaintext and the keystream:

$$C = P \oplus \text{keystream}; \text{ where } \text{keystream} = E_K(\text{IV} \parallel \text{counter})$$

If we reuse an initialization vector under the same key, it causes the block cipher to output an identical keystream. If an attacker intercepts the two ciphertexts  $C_1$  and  $C_2$  that are encrypted under a repeated injection vector, then they can XOR them together:

$$C_1 \oplus C_2 = (P_1 \oplus \text{keystream}) \oplus (P_2 \oplus \text{keystream}) = P_1 \oplus P_2$$

Doing this completely eliminates the layer of encryption, yielding the raw XOR combination of both plaintexts and enabling text recovery. In order to prevent injection vector reuse, the protocol constructs injection vectors dynamically as `session_id || sequence_number`, which guarantees distinct injection vectors for every frame.

The architecture implements “the encrypt then MAC” design, which enforces constant-time HMAC verification over the header, injection vector, and ciphertext before invoking decryption. MAC-and-Encrypt constructions process unauthenticated ciphertexts through the decryption engine first, exposing systems to padding oracle attacks, decryption timing oracles, and memory corruption exploits. Verifying the MAC tag first and failing closed matters because it allows the protocol to discard tampered frames immediately without releasing partial plaintexts or operational error signals.

Independent keys are derived for encryption and message authentication because using a single key across two distinct algorithms risks cross-primitive key interactions. Deriving distinct directional key sets also prevents cross channel key reuse and ensures that compromising or reflecting traffic in one direction does not compromise the other. In contrast to this protocol's use of “encrypt then MAC,” modern security standards use native authenticated encryption with associated data (AEAD) primitives like AES-GCM. AES-GCM combines CTR mode encryption with GHASH into a single algorithm pass. The AEAD primitives eliminate manual composition errors, however, AES-GCM remains fragile to injection vector / nonce reuse while composite “encrypt then MAC” provides modular separation across cryptographic building blocks.

The implemented protections on the transport layer guarantee confidentiality, integrity, and authenticity. This level of security does not evaluate whether the intent of a command is safe, maintaining the risk of unauthorized tool execution. A cryptographically valid record authenticated with a legitimate MAC tag may still transmit a harmful instruction like `{"action": "DELETE", "path": "/etc/passwd"}` if the client is misconfigured or tricked via prompt injection.

The encryption we implement is there to confirm *who* sent a message and that it arrived uncorrupted, but it does not guarantee the same protection as application-level authorization, input sanitization, and access control policies